

浙江理工大学科技与艺术学院

KEYI COLLEGE OF ZHEJIANG SCI-TECH UNIVERSITY

# Linux 操作课程设计

## 实训过程报告

项目名称：盐水与时间（Salt & Time）发酵记录本

班　　级：24 大数据管理与应用

学生姓名：杨　　济　　同

学　　号：Xc2459010 4

指导教师：叶　　杰　　锋

# 1 需求分析

这个应用是从我自己的麻烦来的。家里常年养着泡菜、康普茶、味噌几坛，起坛日期、盐度、计划天数记在手机备忘录和坛口胶带纸上，两三个月就对不上账——有一坛康普茶超期四天才想起来，味道已经过了。

需求很简单：一坛一条档案，录一次；每天记一条观察；偶尔改状态。零散的日期换成“第几天”“还剩几天”“超期几天”，这个数不自己算。

建坛：名字、类型、容器、盐度、开始日期、计划天数；天数换算：第几天、不落库，避免与实际脱节；观测记录：一坛下面挂多条，每条记日期、温度、pH、气味：发酵中、已收、失败三种；失败也留存，便于回溯；筛选与搜索：按状态切（全部、在发酵、可以开罐、已收、失败），按类型筛，按文字搜名字和备注。

实测：① 浏览器打开即用，不注册、不登录；② 新增、查询、修改、删除四类操作都能在界面上完成；③ 必填项为空、盐度超过 30%、pH 超过 14、状态填非法值时必须给出明确报错，不能静默失败；④ 容器重启之后原有数据一条不少；⑤ 宿主机只对外开放一个端口，数据库端口不对外；⑥ 界面不引用外部 CDN，断网也能正常打开。

表 1 坛子表 batches 主要字段

| 字段 | 类型 | 约束 / 默认值 | 说明 |
|---|---|---|---|
| id | BIGSERIAL | 主键 | 自增编号 |
| name | TEXT | 非空 | 坛子名字 |
| kind | TEXT | 非空 | 泡菜 / 酸面团 / 康普茶 / 味噌 / 腌菜 |
| vessel | TEXT | 可空 | 容器与容量 |
| salt_pct | NUMERIC(4,2) | 空或 0～30 | 盐度百分比，酸面团可留空 |
| started_on | DATE | 默认当天 | 起坛日期 |
| target_days | INTEGER | 非空，1～3650 | 计划天数 |
| status | TEXT | fermenting / done / failed | 状态 |
| note | TEXT | 可空 | 备注 |

表 2 观测表 readings 主要字段

| 字段 | 类型 | 约束 / 默认值 | 说明 |
|---|---|---|---|
| id | BIGSERIAL | 主键 | 自增编号 |

| 字段 | 类型 | 约束 / 默认值 | 说明 |
|---|---|---|---|
| batch_id | BIGINT | 外键，级联删除 | 所属坛子 |
| taken_on | DATE | 默认当天 | 观测日期 |
| temp_c | NUMERIC(4,1) | 空或 −20～60 | 温度 |
| ph | NUMERIC(3,2) | 空或 0～14 | pH 值 |
| aroma | TEXT | 可空 | 气味描述 |
| note | TEXT | 可空 | 备注 |

# 2 设计思路

题目没有限定技术栈，我按“能少一个组件就少一个”的原则定方案。

## 2.1 技术方案

后端是 Node 单文件 server.js（约五百行），只依赖 pg，没有 Express、ORM 和打包步骤；前端是三个原生文件，手写 DOM，不引用 CDN；数据库用 PostgreSQL 16 alpine。

这么选有三个考虑：构建快，只装一个包，十几秒完；排查面小，出问题只看一个文件；断网可用，不会因为拉不到字体白屏。另外范围约束写成库里的 CHECK，应用层漏了还能拦一道。

表 3 开发与部署环境

| 项目 | 实际情况 |
|---|---|
| 操作系统 | CentOS Stream 9（内核 5.14.0-617.el9） |
| 服务器配置 | 4 核，3.5 GB 内存，根分区 37 GB |
| 主机地址 | 192.168.119.128（ens160） |
| 容器运行时 | Docker 29.9.2，Docker Compose v5.6.1 |
| 管理面板 | 1Panel v2.4.1，端口 21168 |
| 开发助手 | Hermes Agent v0.22.3，模型 DeepSeek deepseek-v4-flash |
| 部署目录 | /opt/1panel/docker/compose/salt-time |

## 2.2 数据库设计

readings.batch_id 外键指向 batches.id，设 ON DELETE CASCADE，删一坛时观测一起删；合法范围写成表级 CHECK，应用层再校验一遍作双保险。

索引只加一个：readings 上按 (batch_id, taken_on DESC)，对应详情页按坛取最近几条观测的查询。

## 2.3 部署结构

端口、网络、数据卷、日志是这套部署的重点，结构见图 1。

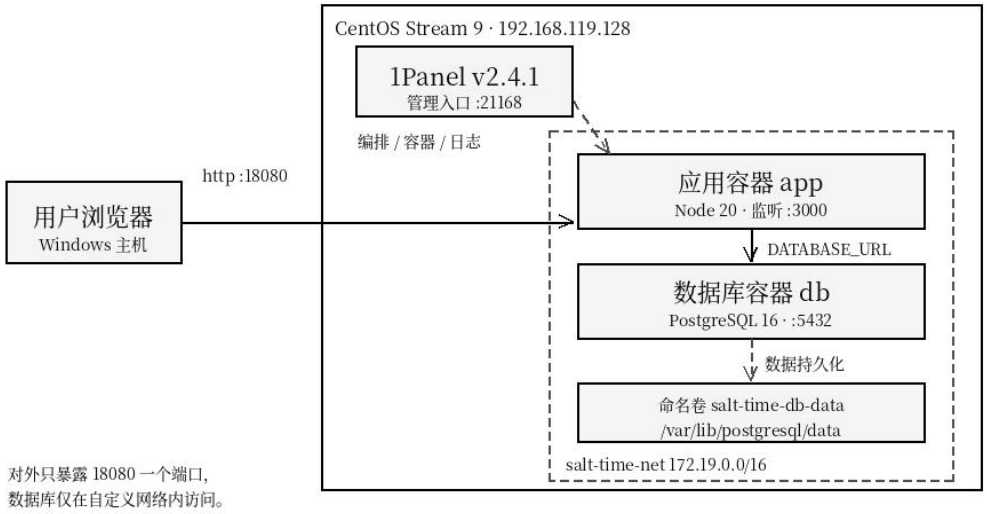



图 1 应用与数据库的部署结构

对外只映射 18080→3000；db 的 5432 不映射到宿主机，只能在 salt-time-net 里用服务名 db 访问。即便以后放到公网，数据库也不会暴露在端口扫描里。

日志用 json-file，单文件 10 MB、最多 4 个。Docker 默认不限大小，跑久了会把磁盘写满。健康检查 db 用 pg_isready，app 用 /healthz；app 等库真连上才启动。

# 3 详细设计

## 3.1 1Panel 安装与初始管理

先确认系统与网络：CentOS Stream 9，内核 5.14.0-617，地址 192.168.119.128/24。

```
[root@localhost salt-time]# uname -r && cat /etc/redhat-release
5.14.0-617.el9.x86_64
CentOS Stream release 9

[root@localhost salt-time]# hostnamectl | grep -vE 'Machine ID|Boot ID|Chassis'
   Static hostname: (unset)
Transient hostname: localhost
         Icon name: computer-vm
    Virtualization: vmware
  Operating System: CentOS Stream 9
       CPE OS Name: cpe:/o:centos:centos:9
            Kernel: Linux 5.14.0-617.el9.x86_64
      Architecture: x86-64
   Hardware Vendor: VMware, Inc.
    Hardware Model: VMware Virtual Platform
  Firmware Version: 6.00

[root@localhost salt-time]# hostname -I
192.168.119.128 172.17.0.1 172.18.0.1 172.19.0.1 240e:473:6230:3643:20c:29ff:fec7:599e

[root@localhost salt-time]# ip -4 addr show ens160 | grep -E 'inet|link'
    inet 192.168.119.128/24 brd 192.168.119.255 scope global dynamic noprefixroute ens160
```

图 2 安装前的系统与网络检查

1Panel 用官方一键脚本安装：

```
bash -c "$(curl -sSL
https://resource.fit2cloud.com/1panel/package/v2/quick_start.sh)"
```

脚本下载 v2.4.1 安装包（65 MB）解压到 /opt/1panel，并生成工具 1pctl。结果见图 3：版本 v2.4.1、端口 21168、安全入口为一串十位随机字符。

```
[root@localhost salt-time]# 1pctl version
------------------------------------------------------------
                      查看 1Panel 版本信息
------------------------------------------------------------
版本: v2.4.1
模式: stable
------------------------------------------------------------

[root@localhost salt-time]# grep -E 'ORIGINAL_(PORT|VERSION|ENTRANCE)' /usr/local/bin/1pctl
ORIGINAL_PORT=21168
ORIGINAL_VERSION=v2.4.1
ORIGINAL_ENTRANCE=3abc9edd18

[root@localhost salt-time]# ss -lntp | grep -E ':21168|:18080' | awk '{print $1,$4,$6}'
LISTEN 0.0.0.0:21168 users:(("1panel-core",pid=4169187,fd=22))
LISTEN 0.0.0.0:18080 users:(("docker-proxy",pid=4073216,fd=8))
LISTEN [::]:18080 users:(("docker-proxy",pid=4073229,fd=8))
```

图 3 1Panel 版本、端口与安全入口

安全入口相当于第二层口令：地址不含它，面板只回一句 Access Temporarily Unavailable，连登录页都不给。

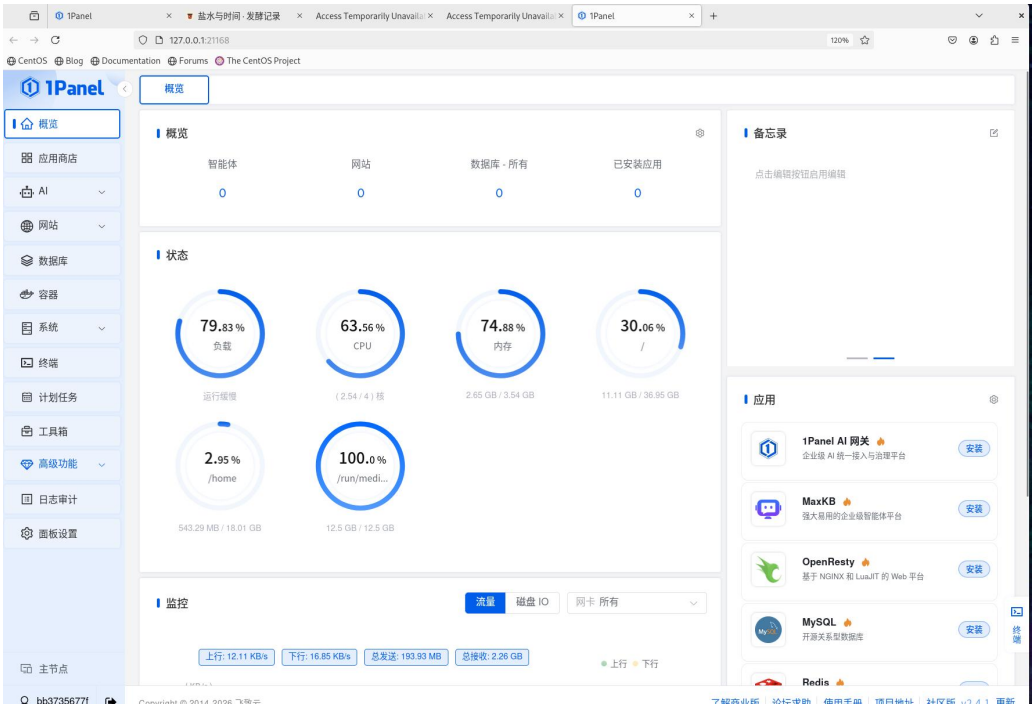



图 4 1Panel 面板概览：负载、CPU、内存与磁盘

面板主要用两处：「主机 → 文件」放项目目录，「主机 → 终端」敲命令；概览页的负载、内存、磁盘一眼可见，排查快。

装完先看资源余量，见图 5。

```
[root@localhost salt-time]# uptime
 14:18:48 up  4:15,  3 users,  load average: 5.01, 4.60, 4.33

[root@localhost salt-time]# free -h
               total        used        free      shared  buff/cache   available
Mem:           3.5Gi       2.6Gi       210Mi        41Mi       1.0Gi       931Mi
Swap:          3.9Gi       1.6Gi       2.3Gi

[root@localhost salt-time]# df -hT | grep -vE 'tmpfs|devtmpfs|overlay|shm'
文件系统           类型      容量  已用  可用 已用% 挂载点
/dev/mapper/cs-root xfs        37G   12G   26G   32% /
/dev/mapper/cs-home xfs        19G  545M   18G    3% /home
/dev/nvme0n1p1      xfs       960M  377M  584M   40% /boot
/dev/sr0            iso9660    13G   13G     0  100% /run/media/ace_c7/CentOS-Stream-9-BaseOS-x86_64

[root@localhost salt-time]# nproc && lscpu | grep -E 'Model name|^CPU\(s\)'
4
BIOS Model name:                      Intel(R) Core(TM) i9-14900HX
```

图 5 服务器负载、内存与磁盘占用

已运行 4 小时 15 分，负载 5.01 / 4.60 / 4.33，4 核机器已过载，但主要是桌面、浏览器和 Hermes 在吃，不是应用容器。内存 3.5 GB 用掉 2.6 GB，根分区用了 32%，所以两个容器都设了上限。

## 3.2 Hermes 安装与执行环境

Hermes 用官方脚本安装：

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
```

装完 hermes --version 为 v0.22.3，git 方式安装，目录 /usr/local/lib/hermes-agent，可执行文件在其 .hermes/bin/ 下。




图 6 Docker 与 Hermes 版本、容器与镜像数量

要写明一点：Hermes 执行后端是 local，terminal 工具直接作用在实训主机，不在容器或沙箱里，输出都是宿主机 root 的。权限很大，所以每次动手前先确认它要改哪个目录。

第一次真实操作是查机器信息（图 2）：主机名 localhost，ens160 上 192.168.119.128/24，4 核，与 1Panel 概览一致，说明面板看的是同一台机器。

## 3.3 模型接口与 API Key 配置

选 DeepSeek：中文提示词理解准、价格低、工具调用格式稳定。

Key 在开放平台创建后写进 /root/.hermes/.env，变量名 DEEPSEEK_API_KEY，另配 DEEPSEEK_BASE_URL；文件权限 600。报告中 Key 一律遮盖。

模型配置在 config.yaml 的 model 段，三行：

```
model:
  provider: "deepseek"
```

```
  default: "deepseek-v4-flash"
  base_url: "https://api.deepseek.com/anthropic"
```

base_url 走 /anthropic 兼容入口，不是 OpenAI 格式，配错会 404 或鉴权失败，所以先发一条最小请求验证再跑真实命令，结果见图 7。

```
[root@localhost salt-time]# grep -nE '^  (provider|default|base_url):' /root/.hermes/config.yaml
75:  default: "deepseek-v4-flash"
108:  provider: "deepseek"
112:  base_url: "https://api.deepseek.com/anthropic"

[root@localhost salt-time]# ls -l /root/.hermes/.env
-rw-------. 1 root root 28428 10月 25 11:55 /root/.hermes/.env

[root@localhost salt-time]# grep -E '^DEEPSEEK' /root/.hermes/.env | sed -E 's/=.*/=**********
    ****/'
DEEPSEEK_API_KEY=********************
DEEPSEEK_BASE_URL=********************

[root@localhost salt-time]# hermes --version | head -1
Hermes Agent v0.22.3 (2026.9.24) · stable · local 5b147ab036d6
```

图 7 Hermes 模型配置与密钥存放位置（密钥已遮盖）

接口可用之后才进入开发。项目代码量不大，flash 版本足够，换更大的模型只会拖慢每一轮修改。

## 3.4 Vibe Coding 开发与迭代

第一条提示词原文照录：

> “用 Hermes 生成容器部署文件，在 1Panel 中部署应用与数据库，检查端口、网络、数据卷和日志，完成基本管理操作。主题可以新颖一点，不要太像 AI 了，并且直接打开就能用，配备完整的 README 及使用教程。”

这条只框了范围和验收口径，没定字段。字段和布局是它给一版、我再改的：盐度允许空着，计划默认 14 天，状态只留三种。

第一次起来页面能开，但有三处不对。都不是功能没写，是边界没处理，且都是点出来的。

一是统计口径。概览写“观测记录 11”，库里只有 9 条：统计接口 join 后 count，一坛多条观测时行数被放大。改成单独 count 即为 9。

二是日期差一天。选 2026-10-21，存进去对，读出来是 10-20：pg 驱动把 DATE 转成 JS Date 再按 UTC 序列化，东八区零点前推一天。改法是统一时区，DATE 按字符串取。

三是弹窗一进页面就开着。面板写了 display: grid，[hidden] 的默认样式被盖掉，加一条 .sheet[hidden] { display: none } 即可。

生成的文件结构见图 8。db/init 下两个 SQL 在卷为空的首次启动时执行；scripts 下三个脚本负责部署、自检和备份。

```
[root@localhost salt-time]# cd /opt/1panel/docker/compose/salt-time && find . -maxdepth 2 -not -path
    './node_modules*' | sort | sed 's|^\./||'
.
db
db/init
docker-compose.yml
Dockerfile
.dockerignore
docs
docs/1panel-部署教程.md
docs/运维手册.md
docs/images
.env
.env.example
.gitignore
LICENSE
package.json
public
public/app.js
public/index.html
public/styles.css
README.md
scripts
scripts/backup.sh
scripts/deploy.sh
scripts/health.sh
server.js
```

图 8 项目目录结构与主要文件

改完把增删改查、筛选、搜索从头点了一遍，界面见图 9 至图 11。




图 9 应用首页：状态计数、概览条与坛子卡片




图 10 坛子详情：观测时间线与新增观测表单

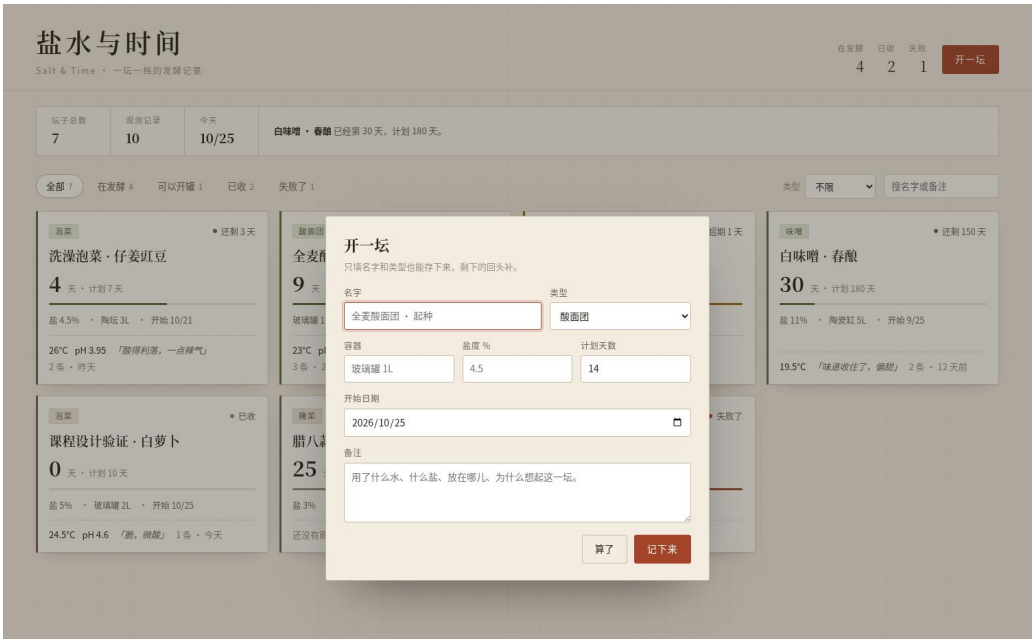



图 11 新建坛子表单与必填项校验

迭代时我不说“感觉不对”，而是把期望值和实际值的差距讲清楚。上面三处都是先给数字（11 对 9、10-21 对 10-20），一次改对。

## 3.5 Docker 部署与 1Panel 管理

Compose 由 Hermes 生成，我改了两处：端口由 8080 换 18080（本机 8080 已占用），并补上日志轮转和内存上限。参数见表 4。

表 4 容器与网络参数

| 项目 | 应用容器 | 数据库容器 |
|---|---|---|
| 服务名 / 容器名 | app / salt-time-app | db / salt-time-db |
| 镜像 | salt-time-app:1.0.0 | postgres:16-alpine |
| 端口映射 | 0.0.0.0:18080 → 3000 | 不对宿主机映射 |
| 网络 | salt-time-net | salt-time-net |
| 数据卷 | 无（无状态） | salt-time-db-data → /var/lib/postgresql/data |
| 依赖关系 | 等 db 健康后再启动 | — |
| 健康检查 | /healthz（含 SELECT 1） | pg_isready |
| 内存上限 | 512 MB | 1 GB |
| 重启策略 | unless-stopped | unless-stopped |

部署先在项目目录执行 docker compose up -d --build；1Panel 把这个目录登记为编排记录，容器页即可启停、看日志。

说明一点：面板管理员密码在 1pctl 里遮蔽显示，我没有猜或改它，编排记录是按面板数据结构登记的，登记前已备份面板数据库。换机器时更稳妥的做法是在「容器 → 编排」里粘贴 compose 内容，效果一样。

```
[root@localhost salt-time]# docker compose ps --format '{{.Name}}   {{.Image}}   {{.Status}}'
salt-time-app   salt-time-app:1.0.0   Up 10 minutes (healthy)
salt-time-db   postgres:16-alpine   Up 10 minutes (healthy)

[root@localhost salt-time]# docker compose ps --format '{{.Name}}   {{.Ports}}'
salt-time-app   0.0.0.0:18080->3000/tcp, [::]:18080->3000/tcp
salt-time-db   5432/tcp

[root@localhost salt-time]# docker images --format '{{.Repository}}:{{.Tag}}   {{.Size}}' | grep -E
    'salt-time|postgres'
salt-time-app:1.0.0   193MB
postgres:16-alpine   418MB
```

图 12 容器运行状态、端口映射与镜像大小

```
[root@localhost salt-time]# docker network inspect salt-time-net -f '{{.Name}}  {{.Driver}}  {{range
     .IPAM.Config}}{{.Subnet}}{{end}}'
salt-time-net  bridge  172.19.0.0/16

[root@localhost salt-time]# docker ps --format '{{.Names}}  {{.Ports}}'
salt-time-app  0.0.0.0:18080->3000/tcp, [::]:18080->3000/tcp
salt-time-db  5432/tcp

[root@localhost salt-time]# docker inspect salt-time-app | grep -m1 '"IPAddress"'
                    "IPAddress": "172.19.0.3",

[root@localhost salt-time]# docker volume ls | grep salt-time
local     salt-time-db-data
```

图 13 自定义网络 salt-time-net 与两个容器的地址

```
[root@localhost salt-time]# docker volume inspect salt-time-db-data --format '{{.Name}} -> {{.Mountp
    oint}}'
salt-time-db-data -> /var/lib/docker/volumes/salt-time-db-data/_data

[root@localhost salt-time]# docker inspect salt-time-db --format '{{range .Mounts}}{{.Type}} {{.Name
    }} -> {{.Destination}}{{println}}{{end}}'
bind  -> /docker-entrypoint-initdb.d
volume salt-time-db-data -> /var/lib/postgresql/data

[root@localhost salt-time]# du -sh /var/lib/docker/volumes/salt-time-db-data/_data
63M⌧/var/lib/docker/volumes/salt-time-db-data/_data
```

图 14 命名卷 salt-time-db-data 的挂载与占用

```
[root@localhost salt-time]# curl -s http://127.0.0.1:18080/healthz
{"ok":true,"db":"up","version":"1.0.0"}

[root@localhost salt-time]# docker inspect -f '{{.Name}} {{.State.Health.Status}}' salt-time-app sal
    t-time-db
/salt-time-app healthy
/salt-time-db healthy

[root@localhost salt-time]# docker logs --tail 6 salt-time-app 2>&1
2026-10-25T06:16:43.667Z 127.0.0.1 GET /healthz 200 26ms
2026-10-25T06:17:14.243Z 127.0.0.1 GET /healthz 200 19ms
2026-10-25T06:17:44.854Z 127.0.0.1 GET /healthz 200 22ms
2026-10-25T06:18:15.403Z 127.0.0.1 GET /healthz 200 20ms
2026-10-25T06:18:45.999Z 127.0.0.1 GET /healthz 200 21ms
2026-10-25T06:18:54.722Z 172.19.0.1 GET /healthz 200 2ms
```

图 15 健康检查结果与应用运行日志

实测值：两个镜像 193 MB、418 MB；网络 172.19.0.0/16，两容器各一地址；db 的 5432 无映射，docker port 查不到；命名卷占 63 MB。

## 3.6 最终验证与运维

验证分七块，均留了命令与输出。

（1）其他设备访问。Windows 浏览器打开 http://192.168.119.128:18080 正常返回，可建坛、看详情；firewalld 已放行 18080。

（2）增删改查。按新建、查单条、加观测、改状态、删除、再查走了一遍：新建 201；观测数 0→1；状态 fermenting→done；删除后返回 ok，再查为 404。见图 16。

```
[root@localhost salt-time]# curl -s -X POST http://127.0.0.1:18080/api/batches -d '{"name":"接口验证
     · 白萝卜","kind":"泡菜",...}'
HTTP 201  ->  id=13  name=接口验证 · 白萝卜  status=fermenting

[root@localhost salt-time]# curl -s http://127.0.0.1:18080/api/batches/13
HTTP 200  ->  id=13  接口验证 · 白萝卜  泡菜  玻璃罐 2L  盐度5%  第0天  观测0条

[root@localhost salt-time]# curl -s -X POST http://127.0.0.1:18080/api/batches/13/readings -d '{"tem
    p_c":24.5,"ph":4.6,"aroma":"脆，微酸"}'
HTTP 201  ->  观测 id=15  温度24.5℃  pH4.6  该坛观测数=1

[root@localhost salt-time]# curl -s -X PATCH http://127.0.0.1:18080/api/batches/13 -d '{"status":"do
    ne"}'
HTTP 200  ->  status=fermenting → done

[root@localhost salt-time]# curl -s -X DELETE http://127.0.0.1:18080/api/batches/13
HTTP 200  ->  {"ok": true, "deleted": 13}

[root@localhost salt-time]# curl -s http://127.0.0.1:18080/api/batches/13    # 删除后再查
HTTP 404  ->  {"error": "没找到这一坛"}

[root@localhost salt-time]# curl -s http://127.0.0.1:18080/api/stats
HTTP 200  ->  total=7  fermenting=4  done=2  failed=1  readings=10
```

图 16 接口层面的增、查、改、删与统计复核

（3）边界与异常。空名字、盐度 45%、pH 15、状态 rotten、目标天数 0、不存在的 id、PUT 请求，分别返回 400、400、400、400、400、404、405，均为中文提示。

```
空名字          POST   /api/batches                -> HTTP 400  {"error":"名字 必填"}
盐度 45        POST   /api/batches               -> HTTP 400  {"error":"类型 必填"}
非法状态         PATCH  /api/batches/9              -> HTTP 400  {"error":"状态只能是 fermenting /
  done / failed"}
pH 15        POST   /api/batches/9/readings      -> HTTP 400  {"error":"pH 不能大于 14"}
目标天数 0       POST   /api/batches                -> HTTP 400  {"error":"类型 必填"}
不存在的 ID      GET    /api/batches/999999         -> HTTP 404  {"error":"没找到这一坛"}
不支持的方法       PUT    /api/batches                -> HTTP 405  {"error":"不支持的方法"}
```

图 17 接口异常输入的返回码与提示

一处瑕疵：盐度超范围和目标天数为 0 返回的是“类型 必填”，因为必填校验先跑，轮不到范围检查。功能无碍，提示可再改一版。

（4）重启持久化。以 id=9 对比：重启前 status=done、盐度 5%、温度 24.5 ℃、pH 4.6；restart 后两容器重建，启动时间 04:18:11→06:01:44，第 4 次轮询均 healthy；再查字段完全一致。

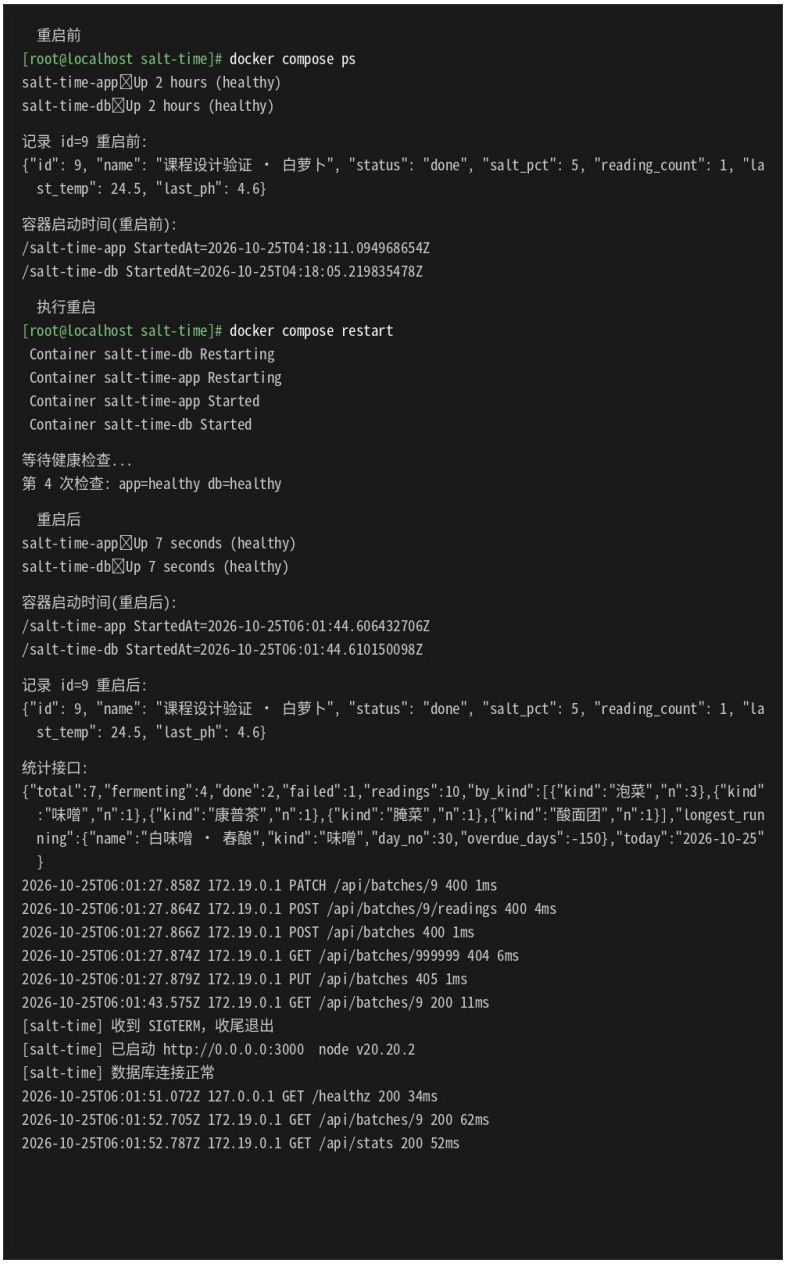



图 18 重启前后同一条记录的字段对比与容器启动时间

（5）备份与恢复。pg_dump 用 -Fc 生成 8543 字节 dump。恢复不覆盖生产库，另建 salt_time_verify 测试库导入对数：两边都是 7 坛 10 条，抽查字段一致；核完 drop 测试库。

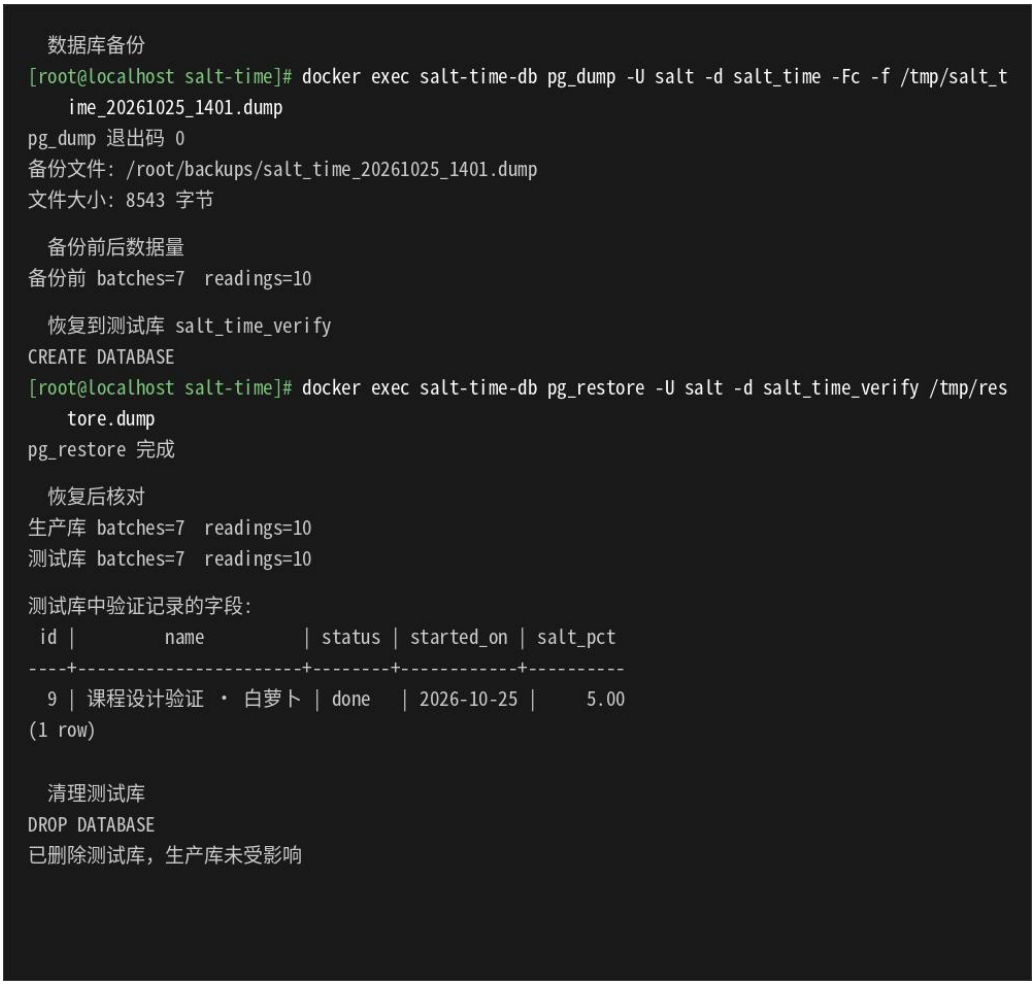



图 19 数据库备份、恢复到测试库与数据核对

（6）版本回退。标签取自 .env 的 APP_VERSION。做法是 docker tag 造 0.9.9、改 .env，再用 --force-recreate 重建，inspect 确认运行的是 0.9.9，复测正常后改回 1.0.0。

这一步踩了坑：只改 .env 时 compose 报 Container Running，容器没重建——shell 里 source 过 .env，环境变量优先级高于 .env 文件。unset 后加 --force-recreate 才真正换镜像。

```
  镜像版本与回退
[root@localhost salt-time]# docker images salt-time-app
salt-time-app:1.0.0  6637e28c5209  2 hours ago

[root@localhost salt-time]# docker tag salt-time-app:1.0.0 salt-time-app:0.9.9      # 造一个旧版本标
    签
salt-time-app:0.9.9  6637e28c5209
salt-time-app:1.0.0  6637e28c5209

回退: .env 中 APP_VERSION 改为 0.9.9
APP_VERSION=0.9.9
[root@localhost salt-time]# docker compose up -d --no-build --force-recreate
 Container salt-time-db Recreated
 Container salt-time-app Recreated
 Container salt-time-db Started
 Container salt-time-app Started

回退后实际使用的镜像:
salt-time-app:0.9.9  状态=running
salt-time-app  salt-time-app:0.9.9  Up 10 seconds (healthy)
salt-time-db  postgres:16-alpine  Up 16 seconds (healthy)
回退后接口复测:
{"ok":true,"db":"up","version":"1.0.0"}

确认可用，恢复到 1.0.0
APP_VERSION=1.0.0
[root@localhost salt-time]# docker compose up -d --no-build --force-recreate
 Container salt-time-db Recreated
 Container salt-time-app Recreated
 Container salt-time-db Started
 Container salt-time-app Started
salt-time-app  salt-time-app:1.0.0  Up 10 seconds (healthy)
salt-time-db  postgres:16-alpine  Up 16 seconds (healthy)
现在运行: salt-time-app:1.0.0
已清理临时标签 salt-time-app:0.9.9

最终确认:
{"ok":true,"db":"up","version":"1.0.0"}
```

图 20 按镜像标签回退到 0.9.9 再恢复到 1.0.0

（7）一次问题排查。卡住我的不是应用而是面板：把深链接（形如 /3abc9edd18/hosts/monitor）贴到地址栏，返回的不是登录页，而是整页 Access Temporarily Unavailable。

排查：curl 本机请求同一地址，HTTP 200 但内容就是这页；路径逐级回砍，只有入口路径返回前端页面。原因是后端只在校验到安全入口时返回单页，前端路由要页面跑起来才生效。

结论：面板只能从入口地址进，进去后在页面里点，不要收藏或转发子页面链接。不是故障，但第一次遇到会以为面板挂了。

另外确认：db 的 /docker-entrypoint-initdb.d 是只读挂载（rw=false），初始化 SQL 只在卷为空的首次启动执行，之后改表结构得自己写迁移，不能指望改 SQL 重启生效。

# 4 简要验收

我在 1Panel 容器页把 app 和 db 各重启一次（等价于 docker compose restart），重启后两个都是 healthy，启动时间已更新。

再打开 http://192.168.119.128:18080，7 坛都在。取“课程设计验证 · 白萝卜”改备注，重新查询确认已落库，看完改回原值，见图 21。

```
[root@localhost salt-time]# curl -s http://127.0.0.1:18080/api/batches/9
HTTP 200  ->  id=9  status=done  备注=用于持久化与备份验证  观测=1条

[root@localhost salt-time]# curl -s -X PATCH http://127.0.0.1:18080/api/batches/9 -d '{"note":"验收
    现场改的备注：坛沿水每天换，改成了隔天换"}'
HTTP 200  ->  备注=验收现场改的备注：坛沿水每天换，改成了隔天换

[root@localhost salt-time]# curl -s http://127.0.0.1:18080/api/batches/9      # 重新查询确认已落库
HTTP 200  ->  备注=验收现场改的备注：坛沿水每天换，改成了隔天换

[root@localhost salt-time]# curl -s -X PATCH http://127.0.0.1:18080/api/batches/9 -d '{"note":"井盐
    + 干辣椒 + 一把花椒。坛沿水每天换一次。"}'    # 改回原值
HTTP 200  ->  备注=井盐 + 干辣椒 + 一把花椒。坛沿水每天换一次。
```

图 21 验收现场修改一个字段并重新查询确认

切到“失败”筛选，只剩那条盐度偏低的酸黄瓜，说明筛选生效。




图 22 切换到“失败”筛选后的列表

数据和界面两步都对得上，验收通过。

# 5 心得体会

最大的收获是把“能跑”和“能证明它能跑”当成两件事。应用第一次起来就能用，真正花时间的是验证：重启后数据还在不在、备份能不能恢复、换标签后容器到底换没换。这些不落成文字和截图，等于没做过。

第二是对 Docker 的理解从“会用命令”变成“知道它在哪一层”。以前写 compose 是抄模板，这次把网络、卷、健康检查、日志轮转逐项查了实际值。看完之后，down 和 down -v 的区别就不用背了——一个停容器，一个连数据删。

第三是 AI 写代码的边界。骨架和页面它搭得很快，但“看着没问题、点两下就露馅”的地方得人来发现。这些不算它写错，是它不会像用户那样去用。所以现在让它先写，我一条条点，再拿具体数字回去改。

不足也清楚：单用户、没有开罐提醒、观测不能导出 CSV、备份是手动跑的。继续做的话，先加开盖提醒，再把 pg_dump 挂到 1Panel 计划任务，最后导出数据画发酵曲线。

还有个教训：命令输出最初只留在终端窗口，一关就没了，后来每条另存一份文本，报告里的图才回填得出来。

## 指导教师评价

以下由教师填写。分项勾选用于反馈，最终成绩按得分及核心验收结果确定。

| 评价项目 | 优 | 良 | 中 | 合格 | 不合格 |
| --- | --- | --- | --- | --- | --- |
| 课堂参与与任务推进 | | | | | |
| Hermes 使用与 AI 操作 | | | | | |
| 过程说明与截图质量 | | | | | |
| 部署与数据库验证 | | | | | |
| 运维操作与问题分析 | | | | | |

总评等级：优（ ） 良（ ） 中（ ） 合格（ ） 不合格（ ）。

指导教师签名： 日期：